

DECOMPOSIÇÃO AUTOMATIZADA DE METAS DE ESTUDO: UM ALGORITMO BASEADO EM AGENTES AUTÔNOMOS PARA ORGANIZAÇÃO ACADÊMICA

Alice Soares^a, Ana Clara Peres^a, Caetano Spazzapan^a, Guilherme Ojeda^a, Renzo Talayeh^a

^aUniversidade Católica Dom Bosco, Campo Grande, MS, Brasil

Abstract

Este trabalho apresenta o desenvolvimento de um Planejador Autônomo Acadêmico baseado em regras lógicas e heurísticas simples, voltado à decomposição automatizada de metas de estudo em subtarefas organizadas. A proposta parte do problema recorrente da gestão de tarefas acadêmicas complexas, que envolvem prazos, prioridades, duração estimada e relações de precedência entre etapas. O protótipo foi implementado em Python, com interface de terminal e interface visual em Streamlit, utilizando validação de dados, normalização textual, tabelas de busca, estruturas condicionais, cálculo de urgência, pontuação por prioridade composta e persistência em arquivo JSON. A metodologia envolveu a organização modular do sistema, a definição de regras determinísticas para geração e ordenação de subtarefas e a validação por meio de testes automatizados e simulação de cenários. Os resultados indicam que o sistema é capaz de interpretar diferentes tipos de tarefas, gerar planos coerentes, classificar níveis de urgência e preservar o progresso do usuário em execução local. Conclui-se que o protótipo demonstra, de forma prática e explicável, como conceitos de Lógica Aplicada à IA, planejamento automático e agentes baseados em regras podem ser empregados na construção de uma solução computacional simples para apoio à organização acadêmica.

Keywords: Agentes autônomos, Planejamento automático, Lógica aplicada à IA, Heurísticas, Organização acadêmica

1. Introdução

A crescente sofisticação da Inteligência Artificial (IA) abriu caminho para o desenvolvimento de sistemas que operam com alto grau de autonomia: percebem o ambiente, definem objetivos e executam sequências de ações sem depender de supervisão contínua. Esses sistemas, chamados de agentes autônomos, ocupam posição central na pesquisa atual em IA [1, 2] e têm demonstrado desempenho expressivo em tarefas que exigem planejamento estruturado, encadeamento de decisões e adaptação dinâmica ao contexto [2]. No campo educacional, essa evolução motivou o surgimento de diversas ferramentas inteligentes de suporte ao estudante — entre elas, sistemas tutores adaptativos [1] e assistentes baseados em Modelos de Linguagem de Grande Escala (Large Language Models, LLMs) [3] —, consolidando uma área de pesquisa em franco crescimento [4]. Ainda assim, observa-se que a maior parte dessas iniciativas não incorpora mecanismos formais de raciocínio lógico nem estratégias de planejamento deliberativo voltadas à gestão personalizada das atividades acadêmicas.

No cotidiano universitário, a gestão de tarefas acadêmicas complexas representa um obstáculo recorrente para boa parte dos estudantes. Atividades como a produção de um artigo científico ou a preparação para uma avaliação de Cálculo não se resumem a ações isoladas: são compostas por múltiplas subtarefas interdependentes, cada uma com seus

próprios prazos e níveis de exigência cognitiva. Organizar esse ecossistema não equivale a simplesmente distribuí-lo em um calendário; trata-se de modelar um sistema de restrições lógicas e precedências temporais, no qual janelas de tempo e recursos cognitivos limitados determinam quais sequências de execução são de fato viáveis. A ausência de um plano estruturado nesse processo frequentemente resulta em subestimação do esforço real, procrastinação ou execução das etapas em sequência inadequada — fatores que, em conjunto, deterioram a qualidade do produto final. Resolver esse problema de maneira automática, a partir dos dados informados pelo usuário, enquadra-se no escopo do Planejamento Automático, pois envolve a geração de uma sequência de ações capaz de transformar uma tarefa acadêmica ampla em um conjunto organizado de subtarefas.

Resolver esse problema justifica-se por razões que vão além do interesse técnico imediato. No plano computacional, a decomposição automática de metas educacionais pode se beneficiar da representação formal do conhecimento, de regras heurísticas e da organização sequencial de ações, elementos associados ao planejamento automático em Inteligência Artificial. No contexto deste trabalho, esses princípios são aplicados de forma simplificada por meio de estruturas condicionais, tabelas de busca e funções determinísticas, permitindo transformar tarefas acadêmicas amplas em subtarefas menores e ordenadas. No plano pedagógico,

um agente capaz de estruturar e externalizar o processo de planejamento cumpre o papel de andaime cognitivo (scaffolding) [1]: ao tornar explícita a lógica por trás da organização das tarefas, ele auxilia o estudante a desenvolver uma habilidade metacognitiva que, para muitos, nunca foi formalmente ensinada. No plano social, a adoção de assistentes inteligentes de gestão acadêmica tem potencial para ampliar o acesso à orientação de qualidade, sobretudo em contextos nos quais a alta proporção de alunos por docente inviabiliza um acompanhamento individualizado, favorecendo condições mais equitativas de formação [2].

Diante desse cenário, o presente trabalho propõe o desenvolvimento de um agente planejador acadêmico baseado em regras lógicas e heurísticas simples. Diferenciando-se de assistentes puramente conversacionais, a solução utiliza uma abordagem algorítmica estruturada para decompor tarefas acadêmicas em subtarefas, classificar sua urgência e organizar uma sequência lógica de execução. As contribuições deste estudo estruturam-se em quatro eixos essenciais: (i) a formulação da rotina estudantil como um sistema de precedências e pesos lógicos; (ii) o desenvolvimento de um algoritmo baseado em manipulação de strings para o tratamento de entradas textuais, tabelas de busca para associar tipos de tarefas a modelos de subtarefas e estruturas condicionais para calcular urgência, pontuação e regras específicas de geração do plano; (iii) a implementação de um mecanismo de persistência de estados robusto e tolerante a falhas baseado em arquivos estruturados, capaz de mitigar a corrupção de dados; e (iv) a validação da solução por meio de testes automatizados e simulação de cenários, permitindo verificar o comportamento das principais funções do sistema diante de diferentes combinações de prazo, prioridade, duração e persistência dos dados.

O restante deste artigo organiza-se da seguinte forma: Seção 2 apresenta a fundamentação teórica; a Seção 3 revisa os trabalhos relacionados; a Seção 4 detalha a metodologia do protótipo; a Seção 5 apresenta os resultados; a Seção 6 discute as implicações; e a Seção 7 sintetiza as conclusões.

2. Fundamentação teórica

2.1. O uso de Inteligência Artificial no desempenho de tarefas

De acordo com Stuart Russell e Peter Norvig [1, p. 7], “Definimos a IA como o estudo de agentes que recebem percepções do ambiente e executam ações.” Diante dessa premissa, o planejamento automático (PA) posiciona-se como a subárea da IA dedicada ao aspecto deliberativo desse agente. Em outras palavras, enquanto a percepção capta o estado atual do ambiente, o planejamento é o processo computacional que permite ao agente projetar o futuro, selecionando e sequenciando as ações necessárias para alcançar um objetivo preestabelecido de forma otimizada.

A relevância da Inteligência Artificial nesse ecossistema se mostra no fato de ela estar integrada à otimização de

processos que moldam a tomada de decisão contemporânea. Consequentemente, a arquitetura de agentes inteligentes é o mecanismo que transforma essa demanda de automação em realidade prática, permitindo que tarefas de alta complexidade analítica sejam planejadas e executadas de forma autônoma, eficiente e com mitigação de erros operacionais.

2.2. A Origem do Planejamento Automático

O planejamento automático como campo científico surgiu buscando racionalizar e capacitar as máquinas durante a execução de funções, como pensar em quais instruções fazer e em qual ordem. Partindo desse pressuposto, o GPS (General Problem Solver), desenvolvido por Newell e Simon [14], serviu como marco precursor para um cenário de otimismo na tecnologia ao demonstrar ser possível resolver problemas por meio de uma busca simbólica estruturada.

O problema central que motivou essa mudança no cenário vigente foi a busca por controlar o robô Shakey, desenvolvido no Stanford Research Institute nos anos de 1966 a 1972. O propósito do projeto era criar um autômato que operasse de forma independente em ambientes realistas baseado nos conceitos de Inteligência Artificial. Além disso, a máquina automatizada operava em cinco níveis hierárquicos e tinha como função navegar por salas interligadas, empurrar caixas e acionar interruptores para atingir objetivos definidos por seus operadores humanos. Para isso, era necessário um sistema que não apenas descrevesse o estado atual do ambiente, mas também fosse capaz de prever como esse estado mudaria após cada ação executada [7, p. 3-4].

Foi nesse contexto que o modelo STRIPS (Stanford Research Institute Problem Solver) foi proposto. Conforme descreve Nilsson [7, p. 5], a eficácia do sistema Shakey derivava justamente “das especificações claras desses níveis e de suas interconexões”. No quarto nível dessa hierarquia, o STRIPS era responsável por construir sequências de ações intermediárias necessárias para cumprir uma tarefa, operando, portanto, sobre representações abstratas do mundo, e não sobre comandos físicos diretos. Essa arquitetura em camadas revela um princípio que transcende a robótica: separar o que deve ser feito do como executar cada passo. Isso permite que o planejamento ocorra em um nível de abstração adequado ao problema, sem precisar lidar simultaneamente com os detalhes de execução.

2.3. O Modelo de Representação STRIPS

O STRIPS fundamenta-se em uma representação do mundo por meio de proposições, fatos que podem assumir valores verdadeiros ou falsos. Ademais, o modelo STRIPS representa um problema de planejamento por meio de uma tupla (P, O, I, G) , em que P é um conjunto finito de proposições, O é um conjunto de operadores, I é o estado inicial e G é o conjunto de objetivos a serem satisfeitos [6, p. 36404]. Cada operador é definido por precondições, ou seja, condições que devem ser verdadeiras para que a ação possa ser executada, e por dois conjuntos de efeitos: a lista

de adições, que especifica o que passa a ser verdadeiro após a execução, e a lista de remoções, que indica o que deixa de valer [12].

Conforme formalizado por Tan e Grastien [6, p. 36404], o resultado da aplicação de um operador a um estado S é definido como $Rst(S, o) = (S \cup o_+) \setminus o_-$, desde que as precondições de o sejam satisfeitas em S . Um plano consiste em uma sequência de operadores que transforma o estado inicial em um estado que satisfaz os objetivos [6, p. 36404].

Para ilustrar esse funcionamento, pode-se considerar o exemplo clássico do mundo dos blocos: para mover o bloco A de sobre o bloco B para sobre o bloco C, as precondições exigem que A e C estejam livres no topo e que A esteja sobre B. Após a execução, B torna-se livre, C deixa de estar livre, e A passa a estar sobre C. Esse mecanismo de atualização explícita do estado, gerenciado pelas listas de adição e remoção, representa a solução parcial do chamado *frame problem*, que diz sobre a dificuldade de especificar tudo o que permanece inalterado após uma ação [1, p. 7].

Do ponto de vista da complexidade computacional, o planejamento proposicional STRIPS pertence à classe PSPACE-completo, uma classe consideravelmente mais difícil do que os problemas NP-completos, pois permite que a sequência de ações necessária para atingir um objetivo tenha comprimento exponencial em relação ao tamanho do problema.

Os mesmos princípios lógicos que estruturam o STRIPS fundamentam a implementação do protótipo deste trabalho. As precondições dos operadores STRIPS são condições que devem ser verdadeiras para que uma ação seja executada, e correspondem diretamente às expressões booleanas avaliadas pelas estruturas condicionais *if/elif* do planejador autônomo. Determinar, por exemplo, se uma tarefa deve receber maior urgência é verificar condições como prazo restante, prioridade declarada e duração estimada. Essas condições são avaliadas por estruturas condicionais no planejador, funcionando como precondições simplificadas para a escolha das regras aplicáveis ao plano. Essa conjunção lógica é a mesma operação que o STRIPS realiza ao checar precondições antes de aplicar um operador.

As tabelas de busca (*lookup tables*) implementadas nos dicionários do sistema, que associam cada tipo de tarefa ao seu conjunto de subtarefas, funcionam como a lista de operadores disponíveis: dado um estado de entrada, o sistema recupera diretamente as ações aplicáveis, em tempo constante, sem percorrer longas cadeias de condicionais. A normalização textual realizada antes de qualquer comparação (remoção de acentos, conversão para minúsculas) cumpre a função lógica de garantir que duas representações do mesmo conceito ("Média" e "media", por exemplo) produzam o mesmo valor de verdade na avaliação das condições, evitando inconsistências no fluxo de decisão. Em conjunto, esses mecanismos traduzem o formalismo do planejamento clássico em código executável, mantendo a coerência lógica que o modelo STRIPS exige.

2.4. Planejamento automático, heurísticas e decisões baseadas em regras

A trajetória do planejamento automático mostra que, embora o formalismo STRIPS tenha grande relevância teórica, sua aplicação prática exige restrições de domínio e estratégias heurísticas capazes de reduzir a complexidade da busca. Em vez de explorar livremente todos os estados possíveis de um problema, muitos sistemas de planejamento utilizam regras, pesos e modelos pré-definidos para selecionar ações mais adequadas em contextos específicos. Essa limitação controlada não elimina a lógica do planejamento, mas a torna mais tratável computacionalmente, especialmente quando o objetivo é resolver problemas bem delimitados.

O protótipo desenvolvido neste trabalho segue essa abordagem simplificada. As tarefas acadêmicas não são tratadas como problemas abertos de planejamento clássico, mas como entradas organizadas por tipo, prazo, prioridade e duração estimada. A partir desses atributos, o sistema utiliza tabelas de busca para recuperar modelos de subtarefas e regras heurísticas para calcular urgência, pontuação e ordem de execução. Assim, a contribuição do sistema está em adaptar princípios do planejamento automático a um domínio educacional restrito, produzindo planos explicáveis, previsíveis e coerentes com a lógica de precedências necessária à organização acadêmica.

3. Trabalhos relacionados

A proposta de um agente planejador para organização acadêmica situa-se principalmente na área de planejamento automático em Inteligência Artificial e nas tecnologias de apoio à aprendizagem. Trabalhos sobre agentes baseados em Grandes Modelos de Linguagem (LLMs) também são considerados nesta revisão, não como tecnologia implementada no protótipo, mas como referência comparativa para discutir abordagens mais recentes de decomposição de tarefas e tomada de decisão automatizada. A revisão a seguir adota como critérios de inclusão trabalhos que (a) propõem ou avaliam técnicas de decomposição automática de tarefas, (b) investigam agentes autônomos orientados a objetivos, ou (c) aplicam IA generativa em contextos educacionais; são excluídos trabalhos que tratam de recomendação de conteúdo sem componente de planejamento sequencial ou que abordam tutoria inteligente sem autonomia do agente.

A decomposição de tarefas em subtarefas ordenadas é um problema clássico de IA. Newell e Simon [14], com o General Problem Solver (GPS), foram os primeiros a formalizar a ideia de que um sistema inteligente pode atingir metas complexas por meio de análise meios-fins (*means-ends analysis*), identificando a diferença entre o estado atual e o estado-objetivo e selecionando operadores capazes de reduzi-la. Esse princípio fundador foi aprofundado por Fikes e Nilsson [12], que introduziram o STRIPS (Stanford Research Institute Problem Solver), um planejador que representa o mundo por fórmulas de cálculo de predicados de primeira ordem e encadeia operadores – cada um

com pré-condições, lista de adições e lista de deleções – para transformar um estado inicial em um estado-meta. O STRIPS tornou-se a base formal de praticamente toda a pesquisa subsequente em planejamento clássico e influencia diretamente a estrutura lógica adotada no presente trabalho: a tarefa acadêmica do estudante é modelada como um estado inicial, as subtarefas são operadores com pré-condições (conhecimentos prévios) e efeitos (competências adquiridas), e o plano final é a sequência ordenada que leva ao objetivo de aprendizagem.

A transição do planejamento clássico para agentes autônomos baseados em linguagem natural ganhou impulso com o advento dos LLMs. Wang et al. [3] apresentam uma revisão sistemática abrangente do campo, propondo um framework unificado composto por quatro módulos – perfil (profiling), memória (memory), planejamento (planning) e ação (action) – que sintetiza a arquitetura de dezenas de agentes publicados entre 2021 e 2023. Os autores demonstram que a capacidade de planejamento dos LLMs pode ser potencializada por estratégias como Chain-of-Thought (CoT), Tree of Thoughts (ToT) e ReAct, todas orientadas à decomposição hierárquica de problemas. O trabalho de Wang et al. [3] é diretamente relevante para a presente proposta, pois os módulos de memória e planejamento descritos pelos autores correspondem, respectivamente, ao histórico de progresso do estudante e à geração dinâmica do plano de estudos. Contudo, o survey não trata de aplicações educacionais específicas nem avalia a eficácia dos agentes na redução da carga cognitiva do usuário, lacuna que o presente trabalho visa preencher.

Acharya, Kuppan e Divya [2], em uma revisão abrangente sobre IA Agêntica (Agentic AI), ampliam essa perspectiva ao caracterizar sistemas autônomos capazes de perseguir objetivos complexos de longo prazo com mínima supervisão humana. Os autores distinguem a IA Agêntica da IA tradicional precisamente pela capacidade de operar em ambientes dinâmicos, reconfigurar estratégias em tempo real e gerenciar múltiplos objetivos simultâneos. O trabalho identifica Aprendizado por Reforço Hierárquico (HRL) e arquiteturas modulares orientadas a objetivos como as abordagens mais promissoras para esse paradigma, e aponta aplicações em saúde, finanças e manufatura. Embora os autores não explorem o domínio educacional, a estrutura de sub-metas hierárquicas descrita pelos autores se aproxima conceitualmente da lógica adotada neste trabalho, pois a tarefa principal é decomposta em etapas menores e organizadas em sequência. No entanto, diferentemente dos agentes baseados em LLMs, o protótipo desenvolvido não utiliza modelo generativo: a decomposição ocorre por meio de regras lógicas, tabelas de busca e funções determinísticas implementadas em Python.

No campo específico da interseção entre LLMs e educação, vários trabalhos recentes evidenciam tanto o potencial quanto as limitações dos agentes autônomos. Dan et al. [10], com o EduChat, e Liffiton et al. [13], com o CodeHelp, demonstram que LLMs ajustados para domínios educacionais podem oferecer suporte personalizado, avaliação

formativa e resposta a dúvidas abertas. Contudo, nenhum desses sistemas realiza planejamento sequencial de estudos – eles respondem a perguntas pontuais em vez de produzir um roteiro ordenado de ações. Drori et al. [11] avançam nessa direção ao empregar síntese de programas e few-shot learning para resolver e explicar problemas matemáticos de nível universitário, aproximando-se da ideia de um agente que guia o estudante passo a passo; entretanto, o escopo permanece restrito à resolução de problemas formais, sem abranger a gestão holística da jornada acadêmica.

A contribuição específica do presente trabalho reside na combinação de três elementos: (1) a formalização da tarefa acadêmica como um problema de planejamento inspirado nos princípios GPS/STRIPS; (2) o uso de regras lógicas, tabelas de busca e heurísticas simples para decompor tarefas em subtarefas; e (3) a entrega do plano em uma interface acessível, permitindo ao usuário visualizar, editar, salvar e recuperar o progresso das atividades acadêmicas. Enquanto Wang et al. [3] e Acharya et al. [2] fornecem o arcabouço teórico dos agentes autônomos, e os trabalhos clássicos de Newell e Simon [14] e Fikes e Nilsson [12] fundamentam a lógica de decomposição adotada, nenhum desses esforços converge para um sistema de planejamento acadêmico personalizável e de fácil uso, justificando assim a relevância da contribuição aqui apresentada.

4. Metodologias

4.1. Organização do protótipo

O Planejador Autônomo Acadêmico foi desenvolvido em Python, com o Streamlit como única dependência externa para a construção da interface visual. O projeto foi organizado em módulos com responsabilidades bem definidas, separando claramente os dados, a lógica de processamento, a persistência e as formas de interação com o usuário.

O módulo `models.py` define as três estruturas centrais do sistema usando dataclasses: Tarefa, que armazena os dados fornecidos pelo usuário; Subtarefa, que representa cada etapa gerada pelo planejador; e Plano, que reúne o resultado completo do processamento. Essa separação entre estrutura de dados e lógica de processamento reflete um dos fundamentos da programação estruturada: decompor um problema complexo em partes menores e coesas.

O módulo `validators.py` concentra as funções de validação de entrada. O módulo `planner.py` contém as regras de decisão do sistema. O módulo `storage.py` cuida do salvamento e carregamento em JSON. Por fim, `main.py` e `app.py` implementam, respectivamente, uma interface de terminal e uma interface visual via Streamlit. Outros módulos auxiliares – `ui_subtarefas.py` e `ui_tarefas.py` – reúnem funções de suporte às interfaces, mantidas separadas para facilitar os testes automatizados, enquanto `componentes_subtarefas.py` concentra elementos visuais auxiliares da interface.

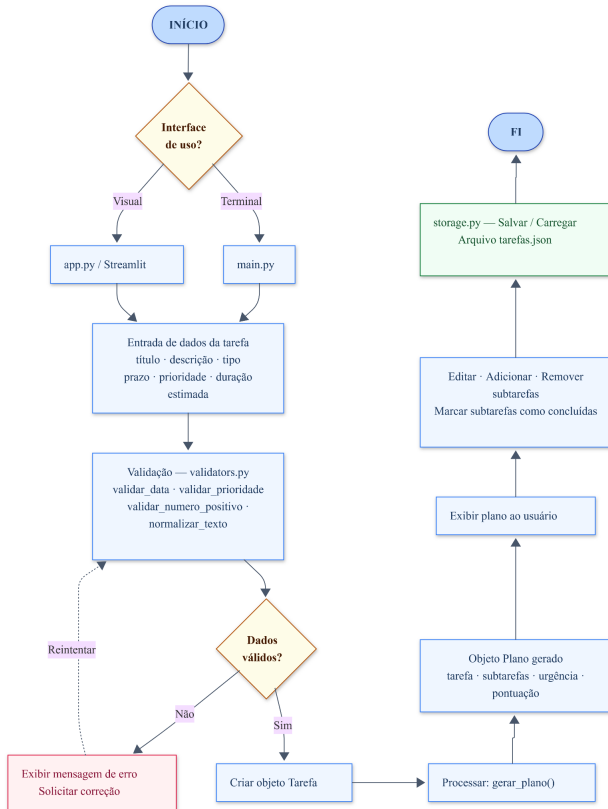


Figura 1: Fluxo geral de funcionamento do Planejador Autônomo Acadêmico.

A Figura 1 apresenta uma visão geral do funcionamento do protótipo, desde a escolha da interface até a geração, edição e persistência do plano em arquivo JSON.

Esse fluxo evidencia a separação entre as etapas de entrada, validação, processamento e armazenamento, reforçando a organização modular adotada no desenvolvimento do sistema.

4.2. Entrada de dados e validação

O usuário pode fornecer os dados da tarefa tanto pela interface de terminal quanto pela interface visual. Em ambos os casos, os campos solicitados são: título da tarefa, descrição opcional, tipo da tarefa, prazo, prioridade e duração estimada em horas.

A validação é realizada por funções específicas em `validators.py`. O prazo é verificado pela função `validar_data`, que tenta converter o texto informado para o formato `dd/mm/aaaa`, retornando falso em caso de falha. A prioridade é verificada pela função `validar_prioridade`, que compara o valor informado com as três opções aceitas: baixa, média ou alta. A duração estimada passa pela função `validar_numero_positivo`, que converte o valor para número e confirma que ele é maior que zero.

O módulo também conta com a função `normalizar_texto`, que remove acentos e converte o texto para letras minúsculas antes das comparações. Esse recurso evita que variações de digitação causem erros nas regras

de decisão — por exemplo, que "Média" e "media" sejam tratadas como entradas diferentes. Todas essas funções utilizam estruturas condicionais e operadores de comparação, retornando valores booleanos que guiam o fluxo do sistema.

4.3. Processamento lógico dos dados

No contexto deste trabalho, o protótipo representa um agente planejador baseado em regras, pois recebe uma tarefa como objetivo, interpreta seus atributos e gera uma sequência de ações por meio de sub-rotinas lógicas.

Após a validação, os dados são processados pela função `gerar_plano`, em `planner.py`. Essa função coordena as demais sub-rotinas em uma sequência lógica definida: identificação do tipo, cálculo de dias restantes, classificação de urgência, cálculo de pontuação, geração das subtarefas, ordenação das subtarefas e geração da justificativa.

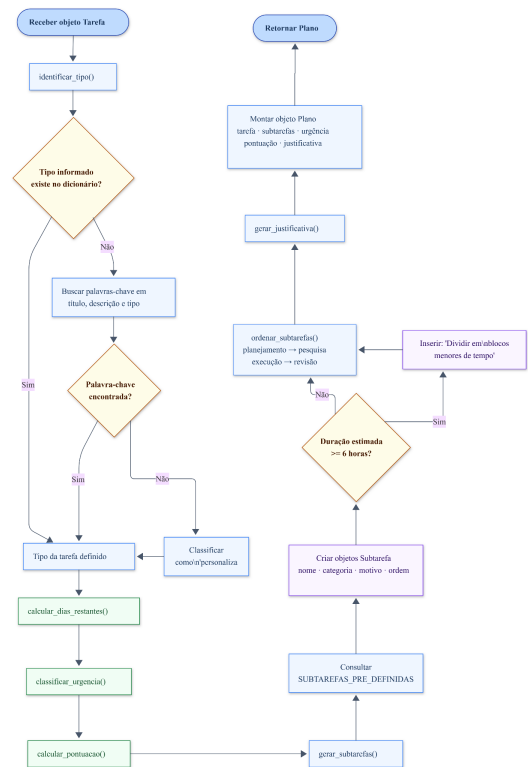


Figura 2: Processamento lógico da função `gerar_plano` no módulo `planner.py`.

A Figura 2 detalha o funcionamento interno da função `gerar_plano`, responsável por transformar os dados da tarefa em um plano estruturado de subtarefas.

Como apresentado no fluxograma, o processamento segue uma sequência lógica de decisões e sub-rotinas, permitindo que o sistema identifique o tipo da tarefa, calcule sua urgência, gere subtarefas adequadas e organize o plano final.

A identificação do tipo é feita pela função `identificar_tipo`. Ela verifica primeiro se o tipo informado pelo usuário corresponde diretamente a um dos tipos disponíveis. Caso contrário, realiza uma busca de palavras-chave no

título, na descrição e no tipo da tarefa, consultando um dicionário de associações definido no próprio código. Se nenhuma correspondência for encontrada, o tipo é classificado como "personalizada".

O cálculo de dias restantes é feito pela função `calcular_dias_restantes`, que subtrai a data atual da data do prazo. Com esse valor em mãos, a função `classificar_urgencia` aplica uma sequência de estruturas condicionais para enquadrar a tarefa em um dos cinco níveis possíveis: prazo vencido, muito urgente, urgente, moderada ou baixa. A prioridade informada pelo usuário também influencia a classificação: uma tarefa com um dia restante e prioridade alta, por exemplo, é classificada como muito urgente.

A pontuação é calculada pela função `calcular_pontuacao`, que soma valores numéricos com base em três fatores: a prioridade (baixa vale 10 pontos, média 20 e alta 30), o prazo (quanto menor o prazo, maior o acréscimo, chegando a 50 pontos para prazo vencido) e a duração estimada (tarefas com seis horas ou mais recebem um acréscimo adicional de 10 pontos). O resultado é um número inteiro que representa a prioridade composta da tarefa, permitindo comparar objetivamente demandas diferentes. Trata-se de uma heurística simples baseada em regras: não há aprendizado de máquina envolvido, apenas operadores aritméticos e condicionais aplicados a pesos pré-definidos.

4.4. Geração e organização das subtarefas

A geração das subtarefas é realizada pela função `gerar_subtarefas`. Com base no tipo identificado, o sistema consulta o dicionário `SUBTAREFAS_PRE_DEFINIDAS`, que contém listas de etapas para cada tipo disponível. Cada etapa é definida por um nome, uma categoria e um motivo, e é transformada em um objeto `Subtarefa` com um número de ordem sequencial.

O sistema conta com modelos para seis tipos de tarefas: trabalho acadêmico, estudo para prova, apresentação, projeto de programação, rotina de estudos e personalizada. A seleção do modelo correto é determinada pelo tipo identificado na etapa anterior, por meio de uma busca direta no dicionário.

Existe também uma regra condicional aplicada à duração estimada: se a tarefa tiver seis ou mais horas previstas, o sistema insere automaticamente uma sub tarefa inicial chamada "Dividir a tarefa em blocos menores de tempo", classificada na categoria de planejamento. Essa regra ilustra como uma condição lógica simples pode adaptar a saída do sistema a contextos específicos sem nenhuma intervenção manual.

Após a geração, as subtarefas são organizadas pela função `ordenar_subtarefas`. Ela utiliza um dicionário de pesos por categoria – planejamento recebe peso 1, pesquisa peso 2, execução peso 3 e revisão peso 4 – e ordena a lista com base nesses valores. Isso garante que as etapas sigam sempre a sequência lógica natural de um projeto acadêmico:

primeiro planejar, depois pesquisar, depois executar e, por fim, revisar.

A interface inclui também recursos de edição e organização das subtarefas, implementados em `ui_subtarefas.py`, que permitem ao usuário ajustar manualmente o plano gerado. É possível adicionar novas etapas, editar os campos de uma etapa existente e remover etapas desnecessárias. Todas essas operações produzem uma nova lista, mantendo identificadores únicos para rastreamento do progresso de cada sub tarefa.

4.5. Interface, persistência e testes

O protótipo oferece duas formas de uso. A interface de terminal, implementada em `main.py`, apresenta um menu numerado que permite cadastrar tarefas, listar, gerar planos, salvar, carregar, carregar exemplos do arquivo `sample_data.json` e excluir tarefas. A interface visual, implementada em `app.py` com Streamlit, reúne as mesmas funcionalidades em um formato mais acessível, organizado em abas para geração de plano, visualização de tarefas salvas e informações sobre o projeto. Essa interface foi publicada no Streamlit Community Cloud, o que permite acessá-la pelo navegador sem instalação local e torna possível a demonstração do protótipo em qualquer dispositivo conectado à internet.

A persistência dos dados é feita por leitura e escrita de arquivos no formato JSON, por meio das funções `salvar_tarefas` e `carregar_tarefas` do módulo `storage.py`. O salvamento utiliza um arquivo temporário: os dados são gravados nele primeiro e, somente após a conclusão bem-sucedida, o arquivo definitivo é substituído. Esse mecanismo evita que uma falha durante a gravação deixe o JSON em estado inválido. Ao carregar, o sistema valida cada entrada antes de adicioná-la à lista, descartando registros com campos ausentes ou inconsistentes.

O projeto inclui quatro arquivos de testes automatizados escritos com a biblioteca `unittest`. O arquivo `test_planner.py` verifica as funções de urgência, cálculo de dias restantes, pontuação, geração de subtarefas e ordenação. O arquivo `test_storage.py` cobre cenários de JSON corrompido, campos inválidos, normalização de progresso e compatibilidade com dados de versões anteriores do sistema. O arquivo `test_interfaces.py` testa exclusão de tarefas via terminal, ordenação e filtragem da listagem, verificação de conclusão total e atualização dos dados de planejamento. O arquivo `test_ui_subtarefas.py` testa as operações de edição manual das subtarefas.

5. Resultados

Os cenários a seguir foram elaborados com base nas funcionalidades observadas no protótipo e nos dados de exemplo disponíveis no projeto. Os valores numéricos apresentados foram calculados a partir das regras implementadas em `planner.py`. A tabela abaixo resume os quatro cenários; cada um é detalhado nas subseções seguintes.

Tabela 1: Cenários, entradas, processamento lógico e saída observada.

CENÁRIO	ENTRADA FORNECIDA	PROCESSAMENTO LÓGICO	SAÍDA OBSERVADA
1 - Trabalho acadêmico	Tipo: trabalho acadêmico; prazo: 07/06/2026; prioridade: alta; duração: 9,5h	Identificação direta do tipo; regra de duração longa ativada ($\geq 6h$); pontuação: 30 (prioridade) + peso do prazo + 10 (duração)	11 subtarefas geradas, incluindo "Dividir em blocos"; urgência calculada conforme o prazo; subtarefas ordenadas por categoria
2 - Estudo para prova	Tipo: estudar para prova; prazo: 12/06/2026; prioridade: média; duração: 4h	Identificação do tipo; 6 dias restantes confirmados pelo teste; pontuação total: $40 = 20$ pontos da prioridade média + 20 pontos do prazo	7 subtarefas; sequência planejamento $\rightarrow$ execução $\rightarrow$ revisão; regra de duração longa não ativada
3 - Prazo iminente	Prioridade: alta; 1 dia restante	Condiçionais de urgência aplicadas em sequência; prioridade alta + 1 dia resulta em "muito urgente"; pontuação: $30 + 40$	Urgência: muito urgente; pontuação mínima de 70; justificativa textual gerada; comportamento confirmado por teste automatizado
4 - Persistência	Tarefas com subtarefas concluídas salvas em JSON	Gravação via arquivo temporário; carregamento com validação campo a campo; JSON corrompido retorna lista vazia	Progresso recuperado corretamente na sessão seguinte; comportamento confirmado por test_storage.py

Figura 3: Interface de entrada de dados da tarefa no protótipo.

A Figura 3 representa o formulário em estado inicial, com valores padrão da interface. Os dados específicos utilizados no Cenário 1 são apresentados na Figura 4.

5.1. Cenário 1 – Trabalho acadêmico com alta prioridade e duração longa

A primeira tarefa do arquivo sample_data.json tem o título "Fazer trabalho acadêmico de Lógica Aplicada", tipo "trabalho acadêmico", prazo para 07 de junho de 2026, prioridade alta e duração estimada de nove horas e meia. As capturas de tela referentes ao Cenário 1 foram obtidas em 06/06/2026, um dia antes do prazo informado. Por esse motivo, o sistema exibe um dia restante na Figura 5.

Com esses dados, o sistema identifica o tipo diretamente pela correspondência com a chave "trabalho acadêmico" no dicionário de modelos. Em seguida, como a duração estimada é de 9,5 horas – valor superior ao limiar de seis –, a regra condicional presente em gerar_subtarefas() insere automaticamente a subtarefa "Dividir a tarefa em blocos" no início da lista. As demais etapas

seguem o modelo pré-definido para esse tipo: entender o tema proposto, definir o problema de pesquisa, buscar referências bibliográficas, organizar a estrutura do trabalho, escrever a introdução, a fundamentação teórica e a metodologia, desenvolver ou explicar o protótipo, analisar os resultados e, por fim, revisar e entregar.

Figura 4: Dados preenchidos para geração do plano acadêmico

A Figura 4 apresenta os dados inseridos no formulário antes da geração do plano, incluindo tipo da tarefa, prioridade, prazo e duração estimada.



Figura 5: Resultado geral do planejamento para tarefa acadêmica com alta prioridade e longa duração.

Ordem	Subtarefa	Categoria	Motivo
1	Dividir a tarefa em blocos menores de tempo.	Planejamento	Como a duração estimada é alta, blocos menores tornam a execução mais controlável.
2	Entender o tema proposto.	Planejamento	Antes de pesquisar, é preciso compreender o que foi pedido.
3	Definir problema de pesquisa.	Planejamento	O problema orienta o restante do trabalho.
4	Organizar estrutura do trabalho.	Planejamento	Uma estrutura evita escrita desordenada.
5	Buscar referências bibliográficas.	Pesquisa	As referências sustentam a fundamentação teórica.

Figura 6: Primeira parte da lista de subtarefas geradas pelo planejador.

6	Escrever introdução.	Execução	A introdução apresenta tema, objetivo e contexto.
7	Escrever fundamentação teórica.	Execução	A teoria explica os conceitos usados no trabalho.
8	Descrever metodologia.	Execução	A metodologia mostra como o trabalho foi desenvolvido.
9	Desenvolver ou explicar protótipo.	Execução	O protótipo demonstra a aplicação prática da ideia.
10	Analisar resultados.	Revisão	A análise interpreta o que foi produzido.
11	Revisar e entregar.	Revisão	A revisão reduz erros antes da entrega final.

Figura 7: Continuação da lista de subtarefas geradas pelo planejador.

Após a ordenação por categoria, todas as etapas de planejamento aparecem antes das de execução, que antecedem as de revisão. A pontuação é composta por 30 pontos da prioridade alta, mais o peso calculado para o prazo informado, mais os 10 pontos extras pela duração longa.

5.2. Cenário 2 – Estudo para prova com prazo moderado

A segunda tarefa do arquivo de exemplo tem o título "Prova de Estatística", tipo "estudar para prova", prazo

para 12 de junho de 2026, prioridade média e duração estimada de quatro horas.

O sistema identifica o tipo e recupera o modelo correspondente, composto por sete subtarefas: listar conteúdos cobrados, separar materiais, ler teoria principal, fazer resumo, resolver exercícios, revisar erros e fazer revisão final. Como a duração estimada é de quatro horas, abaixo do limiar de seis, a subtarefa extra de divisão em blocos não é inserida.

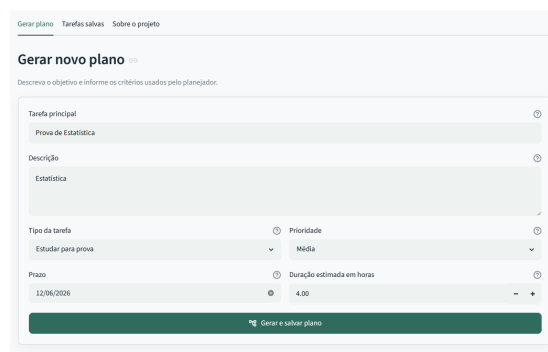


Figura 8: Plano gerado para o cenário de estudo para prova com prazo moderado.



Figura 9: Resultado geral do planejamento para tarefa acadêmica com média prioridade e média duração.

Ordem	Subtarefa	Categoria	Motivo
1	Listar conteúdos cobrados.	Planejamento	Primeiro é preciso saber o que será estudado.
2	Separar materiais.	Planejamento	Materiais organizados reduzem perda de tempo.
3	Ler teoria principal.	Execução	A teoria cria a base de compreensão.
4	Fazer resumo.	Execução	O resumo ajuda a fixar os pontos essenciais.
5	Resolver exercícios.	Execução	Exercícios testam a aplicação do conteúdo.
6	Revisar erros.	Revisão	Corrigir erros evita repeti-los na prova.
7	Fazer revisão final.	Revisão	A revisão final consolida o estudo antes do prazo.

Figura 10: Subtarefas geradas para o cenário de estudo para prova.

A pontuação considera os 20 pontos da prioridade média somados ao peso do prazo. A ordenação coloca as etapas de planejamento primeiro, as de execução no meio e as de revisão ao final. O teste automatizado `test_calculo_de_dias_restantes` confirma que, para um prazo de 12 de junho de 2026 utilizado em 06 de junho, o sistema calcula corretamente seis dias restantes.

5.3. Cenário 3 – Tarefa com prazo iminente e prioridade alta

Conforme apresentado anteriormente na Figura 5, o protótipo exibe ao usuário a classificação de urgência, a pontuação calculada e a justificativa textual do plano gerado. Esse mesmo mecanismo é aplicado no cenário de prazo iminente: quando a tarefa possui prioridade alta e prazo muito curto, a função `classificar_urgencia()` enquadra a tarefa como “muito urgente”, enquanto a função `calcular_pontuacao()` atribui maior peso ao prazo e à prioridade.

Ao final do processamento, o sistema gera uma justificativa textual informando ao usuário o tipo identificado, os dias restantes, a urgência classificada e a pontuação obtida. Essa justificativa funciona como uma explicação legível das decisões tomadas pelas regras lógicas, tornando o processo transparente para quem utiliza o sistema.

5.4. Cenário 4 – Persistência e recuperação de tarefas entre sessões

Após gerar um plano e marcar subtarefas como concluídas, o usuário pode salvar o estado atual pela interface de terminal ou pela interface visual. O sistema grava um arquivo `tarefas.json` contendo todos os campos da tarefa, incluindo a lista de subtarefas já concluídas.

A persistência entre sessões é garantida na execução local por meio do arquivo `tarefas.json`. No ambiente do Streamlit Community Cloud, o armazenamento em arquivo é temporário e pode ser reiniciado durante atualizações, reinitializações ou novas implantações da aplicação. Portanto, a persistência em nuvem deve ser compreendida como uma

limitação do protótipo, não como armazenamento permanente.

Na sessão seguinte, a função `carregar_tarefas()` lê o arquivo, valida cada entrada campo a campo e reconstrói a lista em memória com o progresso preservado. O teste `test_salva_progresso_das_subtarefas`, em `test_storage.py`, confirma que uma tarefa com subtarefas marcadas como concluídas é salva e recarregada corretamente, mantendo tanto a lista de etapas concluídas quanto o campo de conclusão da tarefa. Outro teste do mesmo arquivo verifica que um JSON corrompido não interrompe o sistema: a função retorna uma lista vazia e o protótipo segue funcionando normalmente.

```
{
  "titulo": "Trabalho Acadêmico lógica Aplicada",
  "tipo": "trabalho academico",
  "prazo": "07/06/2026",
  "prioridade": "alta",
  "duracao_estimada": 9.5,
  "descricao": "Planejador Autônomo Acadêmico",
  "subtarefas_concluidas": [
    "Dividir a tarefa em blocos menores de tempo.",
    "Entender o tema proposto."
  ],
  "concluida": false,
  "subtarefas_personalizadas": null
}
```

Figura 11: Registro local das tarefas salvas no arquivo `tarefas.json`.

A Figura 11 apresenta um trecho do arquivo `tarefas.json` gerado na execução local. O registro mostra que o sistema salva os atributos da tarefa e a lista `subtarefas_concluidas`, permitindo recuperar o estado do planejamento em uma sessão posterior.

Na execução local, foi possível verificar a criação do arquivo `tarefas.json`, contendo os dados das tarefas cadastradas e o progresso das subtarefas. Conforme apresentado na Figura 11, o arquivo registra não apenas os atributos da tarefa, mas também a lista `subtarefas_concluidas`, permitindo recuperar o estado do planejamento em uma sessão posterior.

6. Discussão

A análise dos resultados permite interpretar o comportamento do protótipo em quatro dimensões: validade e limites da heurística de priorização, confronto com a complexidade teórica do planejamento, posicionamento frente à literatura revisada e implicações éticas das escolhas arquiteturais.

Validade e limites da heurística de priorização

A função `calcular_pontuacao` combina três fatores: prioridade declarada (10, 20 ou 30 pontos), peso inversamente proporcional ao prazo (até 50 pontos para prazo vencido) e acréscimo de 10 pontos para tarefas com duração igual ou superior a seis horas. A lógica é defensável – deadlines acadêmicos são restrições rígidas e irreversíveis, justificando o maior peso possível ao prazo – e o cenário 3 valida empiricamente esse raciocínio: prioridade alta com um dia restante acumula 70 pontos e aciona a classificação “muito urgente”, comportamento confirmado pelo teste `test_calculo_de_urgencia`.

Contudo, a heurística apresenta limitações estruturais. O sistema opera integralmente sobre a prioridade declarada pelo usuário: um estudante que subestime a relevância de uma tarefa receberá um plano sistematicamente inadequado, independentemente da qualidade do algoritmo. Sem acesso ao histórico de desempenho ou mecanismo de retroalimentação, os pesos não se ajustam ao perfil individual. Os seis tipos pré-definidos também não cobrem tarefas interdisciplinares – uma atividade que combine produção textual e implementação de código seria classificada como "personalizada", perdendo a especificidade dos demais modelos. Por fim, os pesos foram estabelecidos a priori, sem validação empírica com usuários reais; não há evidência, no escopo deste trabalho, de que a proporção adotada reflita com precisão o comportamento real de estudantes diante de conflitos de agenda.

Confronto com a literatura de planejamento

Bylander [5] demonstra que o planejamento proposicional STRIPS pertence à classe PSPACE-completo; Tan e Grastien [6, p. 36404] reforçam esse resultado no contexto de inaproximabilidade de metas. O protótipo não reivindica resolver o problema geral: sua eficiência decorre de uma restrição de escopo deliberada – seis tipos pré-definidos, operadores fixos e ordenação por pesos estáticos, sem busca sobre espaço de estados arbitrários. Essa delimitação torna o problema tratável em tempo constante por tipo, de forma análoga aos sistemas industriais que, conforme Russell e Norvig [1], alcançam eficiência prática por representações de domínio restritas. A ordenação das subtarefas por categoria – planejamento, pesquisa, execução e revisão – constitui uma implementação implícita do princípio de precondições do STRIPS: o sistema nunca produz planos com sequência arbitrária, preservando estruturalmente a lógica natural de qualquer projeto acadêmico.

Posicionamento frente aos trabalhos relacionados

O confronto com os sistemas da Seção 3 revela a contribuição específica do protótipo e seus limites comparativos. EduChat [10] e CodeHelp [13] respondem a perguntas pontuais com alta flexibilidade, mas não produzem sequências ordenadas de ações — o protótipo inverte essa relação, oferecendo um roteiro acionável e reproduzível ao custo de menor expressividade. Drori et al. [11] aproximam-se mais da ideia de um agente orientado a ações sequenciais, mas seu escopo permanece restrito à resolução de problemas formais, sem abranger a gestão holística da jornada acadêmica.

A comparação com agentes baseados em LLMs evidencia uma limitação arquitetural do protótipo: por ser baseado em regras determinísticas, ele não interpreta linguagem natural livre nem reconfigura estratégias em tempo real. Em contraste, agentes baseados em LLMs podem adaptar planos com base em feedback acumulado. O protótipo, por sua vez, gera planos estáticos a partir dos dados do cadastro e não incorpora revisão automática diante de mudanças

de contexto – como alteração de prazo ou surgimento de demandas concorrentes. Essa diferença delimita com precisão o espaço de problemas que o sistema resolve e o que permanece aberto para trabalhos futuros.

Implicações éticas e limitações do experimento

Todo o processamento ocorre localmente, com dados persistidos em JSON no dispositivo do usuário, sem transmissão a serviços externos. Esse design atende às preocupações de Muringa [8] sobre controle de dados estudantis e, conforme Li et al. [9], alinha-se aos princípios de soberania de dados e minimização de exposição de informações pessoais identificados como centrais na governança responsável de IA em educação [8, 9]. A explicabilidade algorítmica integral – cada decisão rastreável até a regra condicional que a originou – complementa essa dimensão, impedindo que o sistema opere como caixa-preta sobre dados pessoais.

Cabe registrar, porém, que a interface Streamlit não foi submetida a testes com estudantes em condições reais de uso, deixando usabilidade percebida e aderência ao sistema como questões empiricamente abertas. Um estudo de usabilidade com participantes reais constituiria a principal agenda de trabalhos futuros: permitiria validar os pesos heurísticos adotados, identificar tipos de tarefa ausentes nos modelos pré-definidos e avaliar se a transparência algorítmica é percebida como vantagem ou sobrecarga cognitiva pelo usuário final.

7. Conclusão

O presente trabalho apresentou o desenvolvimento de um Planejador Autônomo Acadêmico baseado em regras lógicas, com o objetivo de decompor tarefas acadêmicas em subtarefas organizadas e sequenciais. O protótipo foi implementado em Python, com interface em terminal e em Streamlit, utilizando validação de dados, classificação de urgência, cálculo de pontuação, geração de subtarefas e persistência em arquivo JSON.

Os resultados demonstraram que o sistema é capaz de receber diferentes tipos de tarefas, identificar seus atributos principais, gerar planos coerentes e preservar o progresso em execução local. Dessa forma, o protótipo atende à proposta de simular o comportamento de um agente planejador, ainda que por meio de uma abordagem determinística e baseada em regras, sem uso de modelos generativos ou aprendizado de máquina.

Como limitações, destacam-se a dependência de tipos de tarefa pré-definidos, a ausência de aprendizado com o histórico do usuário e a persistência temporária no ambiente do Streamlit Community Cloud. Como trabalhos futuros, sugere-se ampliar os modelos de subtarefas, realizar testes com estudantes reais e investigar formas de adaptação automática dos pesos de prioridade, prazo e duração.

No contexto da disciplina de Lógica Aplicada à IA, o desenvolvimento do protótipo permitiu aplicar conceitos fundamentais como estruturas condicionais, operadores

booleanos, tabelas de busca e regras determinísticas na construção de um agente planejador. Dessa forma, o trabalho demonstrou, de maneira prática, como princípios lógicos podem sustentar decisões automatizadas em um sistema computacional simples, explicável e funcional.

8. Referências

- [1] RUSSELL, Stuart; NORVIG, Peter. Artificial Intelligence: A Modern Approach. 3. ed. Upper Saddle River: Pearson, 2010.
- [2] ACHARYA, Deepak Bhaskar; KUPPAN, Karthigeyan; DIVYA, B. Agentic AI: Autonomous Intelligence for Complex Goals — A Comprehensive Survey. IEEE Access, v. 13, p. 18912–18936, jan. 2025. DOI: 10.1109/ACCESS.2025.3532853.
- [3] WANG, Lei et al. A Survey on Large Language Model Based Autonomous Agents. Frontiers of Computer Science, v. 18, n. 6, p. 186345, 2024. DOI: 10.1007/s11704-024-40231-1.
- [4] ZAWACKI-RICHTER, Olaf; MARÍN, Victoria I.; BOND, Melissa; GOUVERNEUR, Franziska. Systematic review of research on artificial intelligence applications in higher education — where are the educators? International Journal of Educational Technology in Higher Education, v. 16, art. 39, 2019. DOI: 10.1186/s41239-019-0171-0.
- [5] BYLANDER, Tom. The computational complexity of propositional STRIPS planning. Artificial Intelligence, v. 69, n. 1–2, p. 165–204, 1994. DOI: 10.1016/0004-3702(94)90081-7.
- [6] TAN, Xing; GRASTIEN, Alban. Inapproximability of STRIPS Planning. In: Proceedings of the AAAI Conference on Artificial Intelligence, v. 40, n. 43, p. 36403–36411, 2026. DOI: 10.1609/aaai.v40i43.40961.
- [7] NILSSON, Nils J. Shakey the Robot. Technical Note 323. Menlo Park: Artificial Intelligence Center, SRI International, 1984.
- [8] MURINGA, T. P. Exploring ethical dilemmas and institutional challenges in AI adoption: a study of South African universities. Frontiers in Education, v. 10, 1628019, 2025.
- [9] LI, Ming et al. A Framework for Developing University Policies on Generative AI Governance: A Cross-national Comparative Study. arXiv preprint arXiv:2504.02636, 2025. DOI: 10.48550/arXiv.2504.02636.
- [10] DAN, Yifan et al. EduChat: A Large-Scale Language Model-Based Chatbot System for Intelligent Education. arXiv preprint arXiv:2308.02773, 2023.
- [11] DRORI, Iddo et al. A Neural Network Solves, Explains, and Generates University Math Problems by Program Synthesis and Few-Shot Learning at Human Level. Proceedings of the National Academy of Sciences, v. 119, n. 32, e2123433119, 2022.
- [12] FIKES, Richard E.; NILSSON, Nils J. STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving. Artificial Intelligence, v. 2, n. 3–4, p. 189–208, 1971.
- [13] LIFFITON, Mark et al. CodeHelp: Using Large Language Models with Guardrails for Scalable Support in Programming Classes. In: Proceedings of the 23rd Koli Calling International Conference on Computing Education Research, 2023.
- [14] NEWELL, Allen; SIMON, Herbert A. Report on a General Problem-Solving Program. In: IFIP Congress, 1959. p. 256–264.